Structure-from-Motion for an Underwater Vehicle

U. Caliskan, H. korib, M. ahmed
Tutors: Valerio Franchi, Nuno Gracias
Underwater Robotics and Computer Vision Project

Abstract

Autonomous underwater vehicles are widely used for visual inspection, seabed mapping, and scientific data collection, but underwater reconstruction remains difficult because of limited visibility, lighting changes, turbidity, and navigation drift. This project implemented and evaluated a loop-closure-aware Structure-from-Motion pipeline on **Stonefish** simulated underwater imagery (Girona 500-compatible ROS topics) with a downlooking camera and inertial navigation sensors. From ROS bag data we estimated a 3,641-frame trajectory, fused 279 COLMAP keyframes with noisy DVL odometry in GTSAM, and reconstructed a dense Poisson mesh of the surveyed scene. The approach extends existing non-incremental factor graph code by integrating COLMAP feature matching, loop closure, GTSAM optimization, sparse-model synchronization, and COLMAP Multi-View Stereo dense reconstruction.

Index Terms— Structure-from-Motion, Underwater Robotics, Girona 500 AUV, GTSAM, COLMAP, Loop Closure, Factor Graphs

1 Introduction

Underwater vehicles are increasingly used to inspect marine environments, archaeological sites, subsea infrastructure, and biological habitats. For these applications, estimating the trajectory of the robot and reconstructing the surrounding scene are essential. However, underwater environments are challenging for vision-based algorithms because images often suffer from low contrast, color attenuation, blur, suspended particles, and repetitive seabed texture. In addition, navigation sensors accumulate drift over long missions, making global consistency difficult without loop closure detection. Structure-from-Motion (SfM) is the problem of recovering camera motion and 3D scene structure from a sequence of images. The experiments in this report use the **Stonefish** underwater robotics simulator: downlooking camera images, simulated DVL/INS odometry, and ground-truth poses are exported in a Girona 500-compatible ROS bag layout for processing. The final goal is to estimate the complete vehicle trajectory and reconstruct a dense model of the observed seabed.

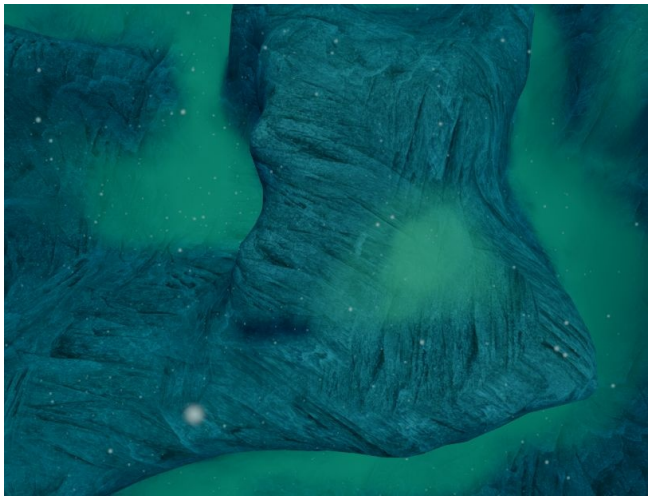


Figure 1: Example Stonefish keyframe (`frame_00087.jpg`): downlooking view of seabed landmarks used for COLMAP feature extraction and sparse reconstruction. The physical Girona 500 platform is not shown; all vision data come from the Stonefish UWR simulation.

1.1 Motivation and Contribution

The motivation for this work comes from the need to obtain accurate underwater maps from real AUV missions. Pure visual reconstruction can fail when image overlap is weak or the scene has few reliable features, while pure navigation estimates can drift over time. Combining visual SfM, inertial/navigation constraints, and loop closure detection can produce a more robust reconstruction pipeline.

Our contribution includes:

1. Adaptation of an existing non-incremental factor graph for joint structure and motion estimation.
2. Implementation of GTSAM factors to optimize the Girona 500 AUV trajectory and 3D landmarks.
3. Use of COLMAP for feature extraction, matching, loop closure detection, dense reconstruction, and texture mapping.
4. Integration of GTSAM and COLMAP into a unified

SfM pipeline using ROS bag data from the underwater mission.

This report presents the implemented underwater SfM pipeline. Section 2 reviews relevant methods and describes the project data. Sections 3 and 4 explain the architecture and implementation. Section 5 reports quantitative and qualitative outcomes on the Girona 500 mission data, and Section 7 concludes the report.

2 Related Work

Structure-from-Motion and visual Simultaneous Localization and Mapping (SLAM) are standard approaches for estimating camera motion and reconstructing 3D structure from image sequences. Classical SfM pipelines rely on feature extraction, descriptor matching, geometric verification, camera pose estimation, triangulation, and bundle adjustment. In underwater robotics, these steps must be adapted to degraded image quality and navigation uncertainty.

Factor graphs provide a flexible way to represent estimation problems in robotics. Vehicle poses, camera poses, and landmarks can be represented as variables and factors. GTSAM is a widely used optimization library for solving such nonlinear least-squares problems, including bundle adjustment and navigation-aided mapping.

COLMAP is a complete SfM and Multi-View Stereo system that provides feature extraction, feature matching, sparse reconstruction, dense stereo, surface reconstruction, and texture mapping. In this project, COLMAP is used not only as a reconstruction tool, but also as a source of loop closure candidates and visual constraints that can be combined with the GTSAM factor graph.

2.1 Key Technologies

Our work builds on three main components. The first component is the Girona 500 ROS bag dataset, which provides the raw camera frames, camera calibration, noisy navigation odometry, and ground truth odometry used to test the pipeline.

The second component is COLMAP, which is used as the visual front-end. Only the extracted image frames and camera calibration are passed to COLMAP in this stage, allowing it to compute feature matches, estimate visual geometry, and detect loop closure candidates between non-consecutive images.

The third component is GTSAM, which is used to formulate and solve the factor graph. The graph combines visual constraints obtained from COLMAP with the noisy odometry from the AUV navigation topic. The optimized trajectory can then be passed back to COLMAP for dense reconstruction, surface reconstruction, and texture mapping.

2.2 Dataset Description

The project data are provided as Girona 500 ROS bag recordings. The original data were generated in ROS1 and converted to ROS2. The bag sequence is split into three files for storage, but the files are processed sequentially according to their index order. Image frames and odometry are extracted and time-synchronized with `extract_gt.py`, producing dense matched CSV files with 3,641 samples per trajectory stream.

The ROS topics used in this work are organized as follows:

- `/girona500/ladybug6/image_color`: raw image frames extracted from the ROS bag and passed to COLMAP.
- `/girona500/ladybug6/camera_info`: camera calibration used to correctly initialize the visual reconstruction.
- `/girona500/navigator/odometry`: noisy odometry used in GTSAM as a motion prior for trajectory optimization.
- `/girona500/dynamics/odometry`: ground truth odometry used only for trajectory evaluation.
- `/girona500/ladybug6/image_color/compressed`: compressed images available as an alternative image source.

Therefore, the processing order is: images and calibration are first passed to COLMAP for visual reconstruction and loop closure detection; the visual constraints and noisy odometry are then optimized in GTSAM; finally, the optimized trajectory is returned to COLMAP for dense reconstruction and texture mapping.

3 Method

3.1 System Overview

The implemented pipeline combines visual information from COLMAP with navigation data from the Girona 500 ROS bag. The raw image frames are first processed in COLMAP to obtain camera poses, feature tracks, and sparse 3D landmarks. These visual outputs are then aligned with the noisy odometry frame and used to build a non-incremental GTSAM factor graph. After optimization, the refined trajectory and landmarks are exported for quantitative analysis and visualization.

Figure 2 summarizes the offline integration loop; the numbered steps below expand the same workflow.

The complete processing chain is organized as follows:

1. Extract and synchronize raw images, noisy odometry, ground truth odometry, and camera calibration from the ROS bag.

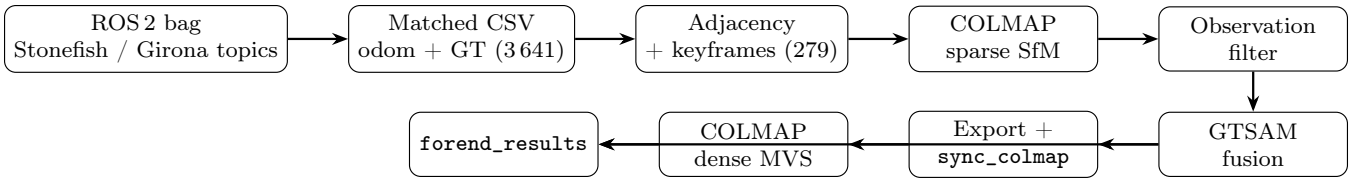


Figure 2: End-to-end pipeline: Stonefish/Girona ROS data → time-synchronized odometry → adjacency-based keyframe selection → COLMAP sparse reconstruction → GTSAM fusion → synchronized COLMAP export → dense reconstruction and diagnostics.

2. Run COLMAP on the extracted image frames to estimate visual camera poses and sparse 3D feature tracks.
3. Align the COLMAP reconstruction to the odometry coordinate frame using a similarity transformation.
4. Build a GTSAM factor graph combining odometry constraints, landmark priors, and robust visual reprojection factors.
5. Optimize the graph with Levenberg–Marquardt and compare the optimized trajectory against ground truth odometry.
6. Export residuals and trajectory data for statistical analysis and 3D visualization.
7. Write the optimized poses and landmarks back into a synchronized COLMAP sparse model.
8. Run COLMAP dense reconstruction and texture mapping to obtain the final dense textured mesh.

For each camera frame i, the vehicle pose is represented as:

$$\mathbf{T}_i = \begin{bmatrix} \mathbf{R}_i & \mathbf{t}_i \\ \mathbf{0}^T & 1 \end{bmatrix} \in SE(3), \tag{1}$$

where $\mathbf{R}_i$ is the orientation and $\mathbf{t}_i$ is the translation. The 3D structure is represented by landmarks $\mathbf{X}_j \in \mathbb{R}^3$ observed by multiple image frames.

3.2 COLMAP Visual Reconstruction

COLMAP provides the visual front-end of the pipeline. From the extracted image sequence, it estimates camera poses and sparse 3D points. The project uses filtered COLMAP files, namely `filtered_images.txt` and `filtered_points3D.txt`. The image file provides the camera pose for each reconstructed frame and the 2D feature observations. The point file provides the corresponding sparse 3D landmark positions.

The sparse reconstruction follows the standard COLMAP front-end. For every input image I_i, scale-invariant (SIFT) features $\mathcal{F}_i$ are extracted. The descriptors are then matched across images: a sequential matcher is used to exploit the temporal ordering of the mission, optionally complemented by exhaustive brute-force matching to recover non-consecutive correspondences. The matched correspondences are passed to the COLMAP `Mapper`, which performs incremental Structure-from-Motion, registering images one by one, triangulating new landmarks, and refining the model with global bundle adjustment. During mapping, loop closures are detected through revisit matching between non-consecutive frames and are integrated into the bundle adjustment, which removes accumulated drift when the vehicle re-observes a previously mapped region.

Because the ROS bag contains thousands of frames, running the full front-end on every image is unnecessarily expensive and produces highly redundant overlap. The reconstruction is therefore organized in two passes. In the first pass, features are extracted and matched, and an *adjacency (co-visibility) matrix* is built in which entry (i, k) counts the number of geometrically verified inlier matches between images i and k. This matrix summarizes the visual connectivity of the sequence and is used to select a reduced, well-connected subset of keyframes that preserves chronological continuity while discarding near-duplicate views. In the second pass, feature extraction and matching are rerun on this downsampled keyframe set, and the mapper produces the final sparse model that is later filtered (Section 3.3) and handed to GTSAM. Figure 3 shows an annotated excerpt of the same match-count matrix stored in `full_adjacency_matrix_grid.csv`: strong matches along the temporal diagonal (sequential overlap) and off-diagonal bands (e.g. loop-style revisits). Keyframe selection on the full 3 641-frame matrix uses `extract_synced_frames.py` and yields 279 views. This procedure is summarized in Algorithm 1.

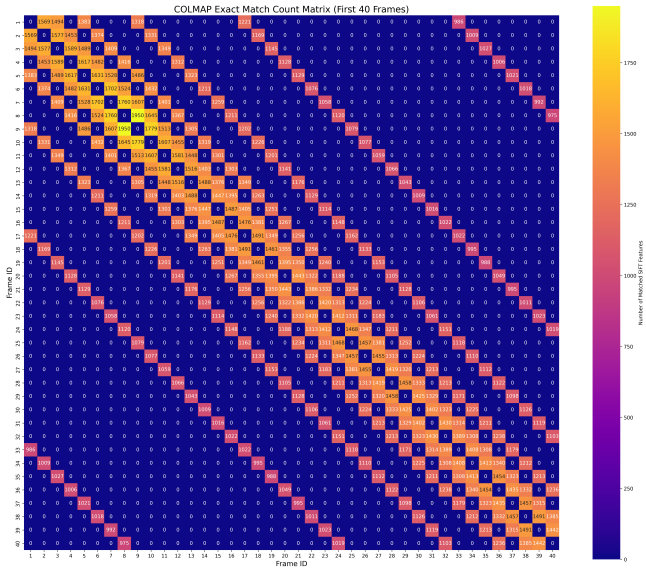


Figure 3: COLMAP exact SIFT match-count matrix (first 40 frames of the mission). Bright cells indicate high co-visibility; off-diagonal structure reveals non-sequential links used for connectivity analysis and keyframe downsampling on the full grid.

Algorithm 1 COLMAP sparse reconstruction with adjacency-based keyframe selection and loop closure

Require: Image set $\mathcal{I} = \{I_1, \ldots, I_N\}$
Ensure: Sparse model $\mathcal{S} = (\mathcal{C}, \mathcal{P}, \mathcal{L})$ ▷ cameras, poses, landmarks
Pass 1 – connectivity analysis:
$\mathcal{F}_i \leftarrow \textsc{FeatureExtract}(I_i)$ for all i ▷ SIFT
$\mathcal{M} \leftarrow \textsc{SequentialMatch}(\mathcal{F})$ ▷ or exhaustive matching
Build adjacency matrix A, $A_{ik} \leftarrow$ # inlier matches (i, k)
$\mathcal{I}' \leftarrow \textsc{SelectKeyframes}(A)$ ▷ downsample, keep overlap + order
Pass 2 – reconstruction:
$\mathcal{F}'_i \leftarrow \textsc{FeatureExtract}(I_i)$ for $I_i \in \mathcal{I}'$
$\mathcal{M}' \leftarrow \textsc{Match}(\mathcal{F}')$
$\mathcal{S} \leftarrow \textsc{Mapper}(\mathcal{I}', \mathcal{M}')$ ▷ incremental SfM + global BA
In $\mathcal{S}$: detect loop closures via revisit matching and refine with bundle adjustment
return $\mathcal{S}$

A 2D image observation is written as:

$$\mathbf{u}_{ij} = (u_{ij}, v_{ij}), \tag{2}$$

where $\mathbf{u}_{ij}$ is the pixel measurement of landmark j in camera frame i. These COLMAP tracks are converted into GTSAM projection factors after invalid observations and missing landmarks are removed.

3.3 Observation Filtering

The raw COLMAP sparse model contains observations that are geometrically unreliable: points that project behind the camera, observations with large reprojection error, and landmarks that are supported by too few views. Feeding these directly into the factor graph would inject outliers into the visual constraints. For this reason, a dedicated pre-GTSAM filtering stage is applied to the COLMAP output before the factor graph is built. This stage is what produces the `filtered_images.txt` and `filtered_points3D.txt` files used by the optimization script.

For each landmark $\mathbf{p}_w$ and each of its observations in image i, the point is first expressed in the camera frame as $\mathbf{p}_c = \mathbf{R}_i \mathbf{p}_w + \mathbf{t}_i$. A *cheirality* test discards the observation if the point lies behind the image plane ($p_{c,z} \leq \tau_z$). The remaining observations are reprojected with the pinhole model $\Pi_K(\cdot)$, and observations whose reprojection error exceeds a pixel threshold ($e > \tau_{px}$) are also marked invalid. Finally, any landmark left with fewer than $L_{\min}$ valid observations is removed entirely, and the image and point files are rewritten with consistent tracks. This procedure is summarized in Algorithm 2.

Algorithm 2 Pre-GTSAM observation filter (cheirality + reprojection)

Require: Sparse model $\mathcal{S}$, thresholds τ_z, τ_{px}, minimum track length $L_{\min}$
Ensure: Filtered sparse model $\mathcal{S}_{\text{filt}}$
for each landmark $\ell \in \mathcal{L}$ with world position $\mathbf{p}_w$ **do**
  for each observation (i, j, u_{ij}, v_{ij}) of ℓ **do**
    Read COLMAP pose $\mathbf{T}_{c \leftarrow w}^{(i)} = (\mathbf{R}_i, \mathbf{t}_i)$
    $\mathbf{p}_c \leftarrow \mathbf{R}_i \mathbf{p}_w + \mathbf{t}_i$
    if $p_{c,z} \leq \tau_z$ **then**
      mark observation invalid; **continue**
    end if
    $(\hat{u}, \hat{v}) \leftarrow \Pi_K(\mathbf{p}_c)$
    $e \leftarrow \|(u_{ij}, v_{ij}) - (\hat{u}, \hat{v})\|_2$
    if $e > \tau_{px}$ **then**
      mark observation invalid
    end if
  end for
  if number of valid observations $< L_{\min}$ **then**
    remove ℓ
  end if
end for
Rewrite `images.txt` and `points3D.txt` with consistent tracks
return $\mathcal{S}_{\text{filt}}$

After this filtering stage, the sparse model retains the most consistent visual landmarks while removing spurious or weakly observed points. In our experiments this

step preserves the large majority of the reconstructed landmarks (29,584 landmarks, about 97.8% of the original sparse points), which indicates that the COLMAP geometry is already well constrained and that filtering mainly removes a small fraction of unreliable observations.

3.4 Coordinate Alignment

COLMAP reconstructs the scene in an arbitrary scale and coordinate frame, while the AUV odometry is expressed in the navigation frame. Therefore, the COLMAP camera centers and landmarks are aligned to the noisy odometry trajectory before graph optimization. A similarity transformation $Sim(3)$ is estimated from the common camera positions:

$$\mathbf{p}^{odom} = s\mathbf{R}_a\mathbf{p}^{colmap} + \mathbf{t}_a, \tag{3}$$

where s is the scale, $\mathbf{R}_a$ is the alignment rotation, and $\mathbf{t}_a$ is the alignment translation. This step allows COLMAP landmarks and visual tracks to be fused with the ROS bag odometry in a common metric frame.

3.5 Factor Graph Formulation

The GTSAM graph contains two types of variables: pose variables $\mathbf{T}_i$ for each camera frame and landmark variables $\mathbf{X}_j$ for each valid COLMAP 3D point. The first pose is fixed with a strong prior to define the local reference frame. Consecutive poses are connected using between factors derived from the noisy odometry. Each valid landmark is initialized from the aligned COLMAP reconstruction and constrained with a weak prior.

Visual measurements are added using calibrated projection factors. The reprojection residual is:

$$\mathbf{e}_{ij} = \mathbf{u}_{ij} - \pi\left(\mathbf{K}, \mathbf{T}_i, \mathbf{T}_{bs}, \mathbf{X}_j\right), \tag{4}$$

where $\mathbf{K}$ is the camera intrinsic matrix, $\mathbf{T}_{bs}$ is the calibrated transform between the vehicle body and the camera sensor, and $\pi(\cdot)$ is the perspective projection model. A Huber robust loss is used to reduce the influence of incorrect visual correspondences. Figure 4 illustrates the variable types and factor families in the graph (poses x_i, landmarks l_j, odometry `BetweenFactorPose3`, and robust projection factors).

We initially attempted `SmartProjectionFactor` factors, which marginalize landmark positions inside the factor and can reduce the number of explicit landmark variables. In our Stonefish underwater sequence this approach was not reliable: optimization repeatedly failed with cheirality-related errors and *degenerate triangulation*. Underwater images often exhibit low parallax (distant features, weak texture, nearly parallel viewing rays), so the triangulation inside smart factors becomes ill-conditioned when camera rays intersect only weakly

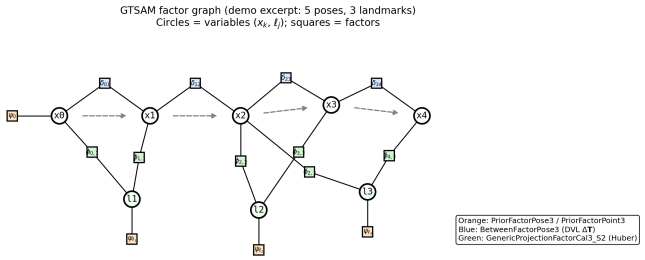


Figure 4: Example GTSAM factor graph structure: pose nodes x_i, landmark nodes l_j, odometry between-factors (blue), landmark priors and pose prior (orange), and `GenericProjectionFactorCal3_S2` visual factors (green, Huber loss).

rather than at a well-constrained 3D point. The final pipeline therefore uses explicit landmark variables l_j with `GenericProjectionFactorCal3_S2` and weak landmark priors, which proved stable for the full graph.

An important implementation detail is the *temporal density* of pose variables versus visual measurements. The factor graph assigns one pose node x_i to every image listed in the dense, time-synchronized odometry CSV (**3,641** frames), connected by odometry `BetweenFactorPose3` constraints along the full mission timeline. Visual reprojection factors, however, are added only for COLMAP keyframes that survived sparse reconstruction and filtering (**279** registered views). Thus GTSAM refines the entire navigation backbone while injecting vision mainly at geometrically well-constrained keyframes.

The optimization objective is:

$$\mathcal{T}^*, \mathcal{X}^* = \arg\min_{\mathcal{T},\mathcal{X}} \sum_i \left\|\mathbf{r}_i^{odom}\right\|^2_{\Sigma_o} + \sum_{(i,j)\in\mathcal{O}} \rho\left(\|\mathbf{e}_{ij}\|^2_{\Sigma_p}\right) + \sum_j \left\|\mathbf{r}_j^{prior}\right\|^2_{\Sigma_l}, \tag{5}$$

where $\mathcal{O}$ is the set of valid image–landmark observations, $\rho(\cdot)$ is the Huber robust loss, Σ_o is the odometry noise, Σ_p is the projection noise, and Σ_l is the landmark prior noise.

3.6 Evaluation and Residual Analysis

After optimization, the local GTSAM poses are transformed back to the global odometry frame. Both the raw noisy trajectory and the optimized trajectory are aligned to the ground truth trajectory before computing the trajectory RMSE. This produces a fair comparison between raw odometry and the GTSAM-fused solution.

For each valid visual observation, the reprojection error is computed before and after optimization. The

exported residual file stores the initial and final pixel errors, the 3D landmark displacement, and the optimized landmark position. These values are used to analyze whether graph optimization reduces visual residuals and stabilizes landmark positions.

3.7 Exporting the Optimized Model to COLMAP

Once the factor graph has been optimized, the refined poses and landmarks must be written back into a COLMAP-compatible sparse model so that the dense reconstruction stage can use them. For each frame, the optimized body pose $\hat{\mathbf{T}}_k$ is composed with the body-to-camera extrinsics and inverted to obtain the camera-to-world transform expected by COLMAP, which is written to `images.txt` as a quaternion and translation. The optimized landmarks, which live in the metric odometry frame, are mapped back into the COLMAP frame using the inverse similarity transform $\mathbf{p}_w = s^{-1}\mathbf{R}_a^\top(\boldsymbol{\ell}^* - \mathbf{t}_a)$ and written to `points3D.txt`.

A key step is re-establishing consistent tracks between the two files. The number of 2D observations per image is recounted, out-of-range or orphaned observations are removed, and landmarks supported by fewer than two views are discarded. An authoritative map from each $(image, feature)$ pair to a landmark identifier is then built from the cleaned point file, and every pixel observation in `images.txt` is re-labelled with the correct `Point3D_ID` (or -1 when it has no valid landmark). This synchronization guarantees that the exported sparse model is internally consistent before dense reconstruction. The same repair logic is available as the standalone script `sync_colmap.py`, which was used to fix `results/sparse/` when COLMAP dense stages reported `point2D.point3D_id` mismatches or out-of-range track indices. The procedure is summarized in Algorithm 3.

Algorithm 3 Export fused solution to COLMAP and model synchronization

Require: Optimized poses $\mathcal{X}^*$, landmarks $\mathcal{L}^*$, alignment $(s, \mathbf{R}_a, \mathbf{t}_a)$, COLMAP files
Ensure: COLMAP-ready sparse folder $\mathcal{S}_{\text{opt}}$

for each frame k with body pose $\hat{\mathbf{T}}_k$ **do**
  $\mathbf{T}_{w\leftarrow c} \leftarrow (\hat{\mathbf{T}}_k \circ \mathbf{T}_{s\leftarrow b})^{-1}$
  Write quaternion and translation to `images.txt`
end for
for each landmark $\boldsymbol{\ell}^*$ in odometry frame **do**
  $\mathbf{p}_w \leftarrow s^{-1}\mathbf{R}_a^\top(\boldsymbol{\ell}^* - \mathbf{t}_a)$ ▷ inverse Sim(3)
  Update `points3D.txt`
end for
$\mathcal{N}_{2D} \leftarrow$ count observations per image from `images.txt`
for each landmark track (i, j) in `points3D.txt` **do**
  if $j \geq \mathcal{N}_{2D}(i)$ **then** remove (i, j) from track
  if track length < 2 **then** remove landmark
end for
Build authoritative map $\mathcal{T}(i, j) \mapsto \ell$ from cleaned `points3D.txt`
for each pixel observation in `images.txt` **do**
  if $(i, j) \in \mathcal{T}$ **then** set `Point3D_ID` $\leftarrow \mathcal{T}(i, j)$ **else** set to -1
end for
return $\mathcal{S}_{\text{opt}}$

3.8 Dense Reconstruction and Texture Mapping

The synchronized sparse model $\mathcal{S}_{\text{opt}}$ is the input to the final dense stage, which is performed entirely within COLMAP's Multi-View Stereo pipeline. First, the input images are undistorted and prepared together with the optimized cameras. COLMAP then runs PatchMatch stereo to estimate a depth map for each view, and these depth maps are merged into a single dense point cloud by stereo fusion. A watertight surface is reconstructed from the fused point cloud using Poisson surface reconstruction, and an optional texture mapping step projects the original images onto the mesh to recover the visual appearance of the seabed. The result is a dense, textured 3D model of the surveyed scene, as summarized in Algorithm 4.

Algorithm 4 COLMAP dense reconstruction on the fused sparse model

Require: Images $\mathcal{I}$, fused sparse model $\mathcal{S}_{\text{opt}}$
Ensure: Dense point cloud and textured mesh $\mathcal{M}$
1: $\mathcal{W} \leftarrow \textsc{ImageUndistorter}(\mathcal{I}, \mathcal{S}_{\text{opt}})$
2: $\mathcal{D} \leftarrow \textsc{PatchMatchStereo}(\mathcal{W})$ ▷ per-view depth maps
3: $\mathcal{P}_{\text{dense}} \leftarrow \textsc{StereoFusion}(\mathcal{D})$
4: $\mathcal{M} \leftarrow \textsc{PoissonMesher}(\mathcal{P}_{\text{dense}})$
5: $\mathcal{M} \leftarrow \textsc{TextureMapping}(\mathcal{M}, \mathcal{W})$ ▷ optional
6: **return** $\mathcal{M}$

4 Implementation

4.1 Software Modules

The end-to-end pipeline is implemented as a chain of Python modules in the `fast_sota` project directory. ROS bag extraction and image–odometry synchronization are performed by `extract_gt.py`, which produces `dense_noisy_matched.csv` and `dense_gt_matched.csv`. Keyframe selection and the adjacency (co-visibility) matrix are built with `extract_synced_frames.py` from the full image set and `full_adjacency_matrix_grid.csv`; the selected views are stored under `keyframes_only/` for COLMAP. The pre-GTSAM observation filter is `colmap_filter.py` (equivalently `filter/observation_filter.py`), yielding `filtered_images.txt` and `filtered_points3D.txt`. GTSAM fusion and COLMAP export are in `backend_new_approach.py`; diagnostics and visualization are in `forend_results.py`. Before dense COLMAP stages, the sparse model in `results/sparse/` is repaired with `sync_colmap.py`, which removes out-of-range track indices and synchronizes `POINT3D_ID` fields between `images.txt` and `points3D.txt`.

4.2 Input Files and Calibration

The optimization stage uses four main input files: `dense_noisy_matched.csv`, `dense_gt_matched.csv`, `filtered_images.txt`, and `filtered_points3D.txt`. The first two files contain the image-synchronized noisy odometry and ground truth odometry for all **3,641** mission frames. The last two files are exported from COLMAP and contain the filtered camera poses, 2D feature tracks, and sparse 3D landmarks for **279** keyframes.

The camera model is initialized using the intrinsic parameters $F_X = 252.4472$, $F_Y = 252.4472$, $C_X = 404.0$, and $C_Y = 308.0$. These values are stored in a GTSAM `Cal3_S2` calibration object. The body-to-camera extrinsic transform is also defined using a small roll, pitch, and yaw rotation and a translation of approximately $(0.000799, -0.001038, 0)$ m. This transform is passed to each projection factor so that image observations are correctly related to the AUV body poses.

4.3 Odometry Loading and Pose Initialization

The noisy and ground truth odometry files are loaded using a dedicated CSV parser. For each image, the parser reads the image name, translation (x, y, z), and quaternion (q_w, q_x, q_y, q_z), and converts them into GTSAM `Pose3` objects. The image order from the noisy odometry file defines the chronological order of the factor graph nodes.

Each image is assigned a pose node x_i. The initial value of each pose is computed from the noisy odometry and expressed relative to the first pose. The first pose is fixed at the origin with a tight prior, which defines the local coordinate system of the graph. Consecutive poses are then connected with `BetweenFactorPose3` factors computed from the relative motion between consecutive noisy odometry poses. This creates a dense odometry backbone for the trajectory.

4.4 COLMAP Pose and Landmark Import

The COLMAP camera poses are read from `filtered_images.txt`. For each reconstructed image, the quaternion and translation are converted into a GTSAM pose. Since COLMAP stores the camera transformation in camera-to-world convention differently from the graph convention, the pose is inverted to obtain the world-to-camera pose used for alignment.

The sparse 3D landmarks are read from `filtered_points3D.txt`. Invalid or incomplete entries are ignored. The remaining points are treated as visual landmarks, referred to as anchor rocks in the diagnostic plots. These landmarks are later inserted into the graph as GTSAM point variables l_j.

4.5 COLMAP-to-Odometry Alignment

Because COLMAP estimates structure up to an arbitrary scale and coordinate frame, the COLMAP reconstruction is aligned to the noisy odometry frame before graph construction. The implementation finds all images that exist in both the COLMAP output and the odometry sequence. It then estimates a similarity transformation using the corresponding COLMAP camera centers and noisy odometry positions.

The alignment estimates a scale s, rotation $\mathbf{R}_a$, and translation $\mathbf{t}_a$ such that:

$$\mathbf{p}_{odom} = s\mathbf{R}_a\mathbf{p}_{colmap} + \mathbf{t}_a. \tag{6}$$

This transformation is applied to every COLMAP 3D landmark. As a result, the visual landmarks, the noisy odometry trajectory, and the GTSAM graph are expressed in a common metric coordinate frame.

4.6 Track Extraction and Filtering

The second line associated with each COLMAP image in `filtered_images.txt` contains the 2D feature observations. The implementation parses these observations as triplets $(u, v, point_id)$. Observations with `point_id=-1` are discarded because they are not associated with a valid 3D point. Observations whose landmark was not loaded from `filtered_points3D.txt` are also removed.

The remaining observations are grouped by landmark identifier. This produces a track dictionary that maps each landmark to all camera nodes where it is observed. This structure is later used to add projection factors between camera poses and landmarks.

4.7 GTSAM Factor Graph Construction

The factor graph contains pose variables x_i and landmark variables l_j. The pose variables are initialized from noisy odometry. The landmark variables are initialized from the transformed COLMAP 3D points. Each landmark also receives a weak 3D prior with an isotropic noise of 1.0 m. This prior keeps landmarks close to their COLMAP initialization while still allowing the optimizer to adjust them.

For the visual measurements, each valid 2D observation is inserted as a `GenericProjectionFactorCal3_S2`. `SmartProjectionFactor` was tested first but abandoned because the optimizer failed with cheirality / degenerate-triangulation errors on many underwater tracks (low parallax and distant features). The generic projection factor keeps each landmark as an explicit variable l_j initialized from COLMAP, which avoids marginalizing poorly conditioned 3D points inside the factor.

Each projection factor connects a camera pose x_i to a landmark l_j using the observed pixel location, the camera intrinsics, and the body-to-camera extrinsic transform. The pixel measurement noise is modeled as an isotropic noise with standard deviation 1.0 pixel. A Huber robust loss with parameter 3.0 is applied to reduce the influence of outlier feature matches.

The pose graph therefore combines three sources of information: a prior on the first pose, relative odometry constraints between consecutive frames, and robust visual reprojection constraints from COLMAP tracks.

4.8 Nonlinear Optimization

The complete graph is optimized using the GTSAM Levenberg–Marquardt optimizer with a maximum of 100 iterations. Before optimization, the implementation prints the initial graph error. After convergence, it prints the final graph error and the number of optimizer iterations. This provides a direct check that the graph optimization has reduced the combined odometry and visual reprojection cost.

4.9 Trajectory Evaluation

After optimization, the local optimized poses are converted back to the global odometry frame by composing them with the first original odometry pose. The implementation then compares three trajectories: ground truth odometry, raw noisy odometry, and the GTSAM-optimized trajectory.

To make the comparison fair, both the raw and optimized trajectories are aligned to the ground truth trajectory using a similarity alignment. The root mean square error (RMSE) is then computed for the raw trajectory and the optimized trajectory. The script also reports the percentage error reduction and the maximum local drift, including the image frame where the largest drift occurs.

4.10 Residual Export

For every valid landmark observation, the implementation computes the reprojection error before and after optimization. If a projection is numerically invalid, the value is marked as `Degenerate`; otherwise, the horizontal and vertical pixel residuals are exported as U and V errors. The script also stores the 3D shift of each landmark between its initial and optimized position.

The residual information is written to `results/exhaustive_residuals.csv`. Each row contains the landmark identifier, camera node, observed pixel coordinates, initial residuals, final residuals, landmark displacement, and optimized landmark position. This file is the basis for the residual plots and landmark analysis.

4.11 Diagnostic Plotting

The second script reads `results/exhaustive_residuals.csv` and `results/trajectory_data.csv`. It first removes degenerate residuals and converts all numeric residual and landmark-shift columns to floating point values. It then computes the reprojection error magnitude before and after optimization:

$$E = \sqrt{U^2 + V^2}. \tag{7}$$

The diagnostic analysis is designed to evaluate both the geometric accuracy of the trajectory and the consistency of the visual landmarks. First, the ground truth, raw odometry, and optimized trajectory are compared in 3D, together with the optimized landmark positions. This gives a global view of how much the GTSAM fusion corrects the noisy navigation estimate. Second, the reprojection residuals are analyzed before and after optimization using scatter plots, sorted error curves, and residual distributions. These plots indicate whether the visual constraints become more consistent after optimization. Finally, an adjacency matrix is used to summarize

the connectivity between camera frames through shared landmarks, giving a topological view of the visual overlap across the sequence.

4.12 Open3D Visualization

The diagnostic script `forend_results.py` can launch an interactive Open3D viewer. By default it draws the ground truth trajectory in black, the raw noisy odometry in blue, and the GTSAM-fused trajectory in red (`SHOW_TRAJECTORIES = True`). Sparse landmark cubes, COLMAP point clouds, and per-frame camera axes are disabled by default (`SHOW_SPARSE_LANDMARKS`, `SHOW_COLMAP_SPARSE`, and `SHOW_CAMERA_FRAMES` set to `False`) so that a dense Poisson mesh from `results/dense/meshed.ply` remains readable. When a dense mesh is loaded, trajectories are transformed into the COLMAP/dense coordinate frame via a Sim(3) fit between fused poses and optimized COLMAP camera centers, which avoids the scale mismatch that otherwise makes the navigation path appear negligible compared with the reconstructed surface.

This viewer is used for qualitative inspection: trajectory shape relative to the dense model, loop-closure consistency, and mesh coverage. Quantitative mesh-to-mesh scores (Hausdorff distance, F-score) are not computed in this project because the Girona 500 ROS bag provides odometry and images but no reference surface mesh.

5 Results and Evaluation

5.1 Evaluation Metrics

The evaluation focuses on the accuracy of the estimated trajectory, the consistency of the visual landmarks, and the quality of the reconstructed scene. The optimized trajectory is compared against the ground truth odometry, while reprojection residuals are analyzed before and after graph optimization. For the reconstructed mesh, geometric distances such as the Hausdorff distance and F-score can be used to quantify surface accuracy and completeness. Table 1 summarizes the main evaluation criteria used in this work.

Table 1: Evaluation Metrics for the Underwater SfM Pipeline

Metric	Description	Status
Trajectory RMSE	Error vs. GT (Sim(3))	`backend_new_approach.py`
Reprojection error	Pixel residual	`exhaustive_residuals.csv`
Landmark retention	COLMAP filter keep rate	97.8% (Table 2)
Dense mesh	Poisson surface	`meshed.ply`
Hausdorff / F-score	Mesh-to-mesh metric	N/A (no GT mesh)

5.2 Quantitative Summary

Table 2 summarizes the main numbers from the current `fast_sota` experiment. Trajectory RMSE values are obtained after aligning each trajectory separately to ground truth with the same Sim(3) procedure as in `backend_new_approach.py`. The optimizer primarily reduces the joint odometry and visual cost; reprojection consistency improves by about two orders of magnitude, while translation RMSE relative to GT can differ from raw DVL because visual factors are sparse in time (279 keyframes vs. 3,641 pose nodes).

Table 2: Main experimental results (Stonefish UWR run, `fast_sota`)

Quantity	Value
Dense odometry frames (CSV)	3 641
COLMAP registered keyframes	279
Sparse landmarks (filtered)	29 584 (97.8%)
Visual observations (GTSAM)	607 697
Mean reprojection error	$22.1 \rightarrow 0.47$ px
Median reprojection error	$19.9 \rightarrow 0.30$ px
Dense fused point cloud	3.67 M
Poisson mesh (vert./faces)	10.6 M / 21.1 M
Raw DVL RMSE vs. GT	3.76 cm
GTSAM fused RMSE vs. GT	8.66 cm

5.3 Trajectory Estimation

Figure 5 compares ground truth odometry (`/girona500/dynamics/odometry`), raw noisy navigation (`/girona500/navigator/odometry`), and the fused trajectory exported to `results/trajectory_data.csv`. The plot is generated by `forend_results.py` and saved as `results/figures/01_trajectory_comparison.png`.

5.4 Sparse Reconstruction and Visual Consistency

The sparse COLMAP model after filtering retains 29 584 landmarks. GTSAM optimization drives the mean reprojection residual from 22.14 px to 0.47 px over 607 697 valid observations, indicating that odometry and vision constraints are brought into strong mutual agreement at the keyframes. Figure 6 shows scatter, sorted, and distribution views of the residuals together with 3D landmark displacements after optimization.

5.5 Dense Reconstruction

Dense reconstruction was run on the synchronized sparse model in `results/sparse/` after `sync_colmap.py` repaired track consistency (required for `image_undistorter` and PatchMatch stereo). COLMAP stereo fusion produced `results/dense/fused.ply` with 3.67 million

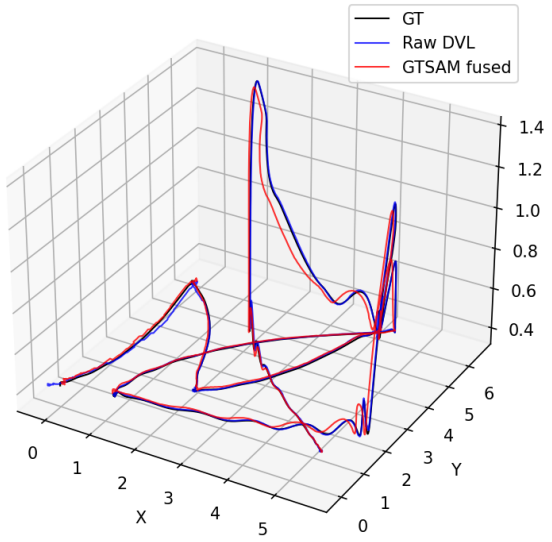
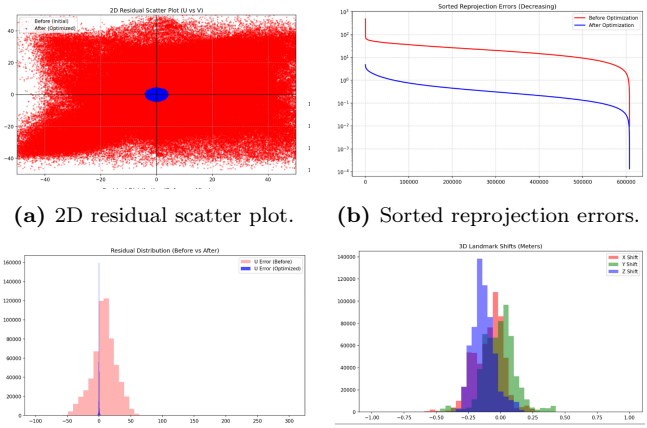


Figure 5: 3D trajectory comparison: ground truth (black), raw DVL odometry (blue), and GTSAM-fused trajectory (red) after export from the factor graph.



(a) 2D residual scatter plot. **(b)** Sorted reprojection errors.

(c) Residual distribution. **(d)** 3D landmark shifts.

Figure 6: Residual analysis plots: (a) 2D residual scatter plot, (b) sorted reprojection errors, (c) residual distribution, and (d) 3D landmark shifts.

points; Poisson surface reconstruction yielded `results/dense/meshed.ply` with about 10.6 million vertices. A non-fatal Poisson warning (`bad average roots: 13`) was observed but did not prevent mesh export. Texture mapping was not required for the quantitative evaluation in this report; qualitative appearance is assessed from the shaded mesh and optional Open3D overlay.

5.6 Challenges Observed

Underwater imagery remains the dominant difficulty: blur, lighting variation, weak texture, and

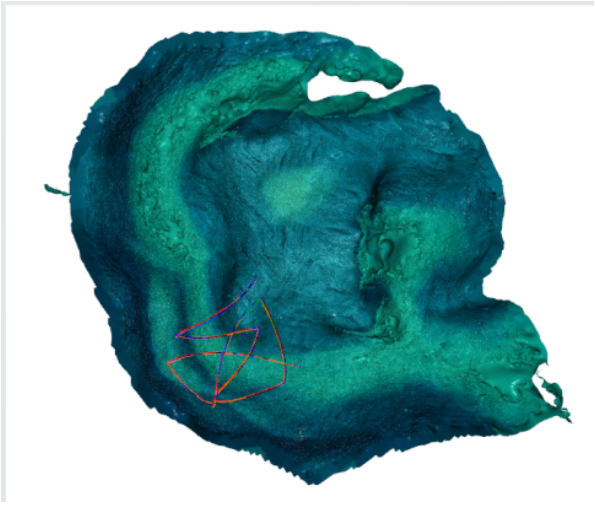


Figure 7: Dense Poisson mesh (`results/dense/meshed.ply`) obtained from the GTSAM-refined sparse model and COLMAP Multi-View Stereo.

repeated seabed patterns can produce incorrect matches or unstable landmarks. Exporting GTSAM results back to COLMAP required explicit sparse-model synchronization—stale `POINT3D_ID` entries and out-of-range track indices otherwise caused `image_undistorter` to abort. The pipeline mitigates visual outliers through cheirality and reprojection filtering, COLMAP geometric verification, Huber robust projection factors, and `sync_colmap.py`. The trade-off between downsampling to 279 keyframes and maintaining a 3,641-frame odometry backbone is discussed in Section 5.7.

5.7 Computational Complexity

The computational cost is mainly determined by the number of selected images, the number of COLMAP feature tracks, and the number of landmarks inserted into the GTSAM graph: feature matching grows with the number of candidate image pairs, while graph optimization grows with the number of poses, odometry factors, landmark variables, and reprojection factors. To control this cost, the dataset is reduced carefully rather than randomly, keeping enough frames to preserve the chronological order of the mission and sufficient visual overlap for continuous motion estimation, while discarding redundant images and unreliable landmarks that would otherwise only add computation. Dense reconstruction and texture mapping remain the most expensive stages, but they are applied only after trajectory optimization, when the camera poses and landmarks are already consistent.

6 Discussion

6.1 Key Findings

The implemented pipeline successfully closes the loop COLMAP → GTSAM → COLMAP: sparse vision and noisy DVL odometry are fused in a single factor graph, the refined model is exported to `results/sparse/`, and dense Multi-View Stereo produces a million-scale surface mesh. The strongest quantitative gain is visual consistency—mean reprojection error drops from 22.14 px to 0.47 px—while the full 3,641-frame trajectory remains connected through odometry factors even though reprojection constraints exist only on 279 COLMAP keyframes. Mesh-to-mesh accuracy metrics were not evaluated because no reference triangle mesh is available in the mission bag.

6.2 Limitations

Although the proposed pipeline provides a structured way to combine COLMAP visual reconstruction with GTSAM trajectory optimization, several limitations remain. First, the quality of the visual constraints depends strongly on the underwater images. In low-texture areas, turbid water, or frames affected by blur and illumination changes, COLMAP may produce fewer reliable matches or unstable landmarks. `SmartProjectionFactor` was not used in the final system because it led to cheirality failures and degenerate triangulation when rays were nearly parallel—a common situation with low parallax and distant seabed features—even though explicit `GenericProjectionFactorCal3_S2` factors remained stable.

Second, loop closure constraints must be handled carefully. If an incorrect loop closure is accepted, it can introduce a wrong geometric constraint and deform the optimized trajectory. This is particularly important in underwater environments, where repeated seabed patterns can make different places look visually similar.

Third, the reconstruction is sensitive to calibration and synchronization. Small errors in camera intrinsics, body-to-camera extrinsics, or image–odometry timestamp association can directly affect the reprojection factors and reduce the quality of the optimized solution.

Finally, dense reconstruction may still contain holes or incomplete regions when image overlap is insufficient, when many frames must be downsampled, or when lighting conditions change significantly along the trajectory. These limitations show that the pipeline depends not only on the optimization method, but also on the quality and continuity of the input data.

6.3 Future Work

Potential future improvements include:

- Adding underwater image enhancement before feature extraction.
- Improving loop closure validation using navigation consistency checks.
- Incorporating depth, altitude, or bathymetric priors when available.
- Testing alternative feature descriptors for underwater imagery.
- Automating the full ROSbag-to-textured-mesh processing workflow.

7 Conclusion

This report presented a Structure-from-Motion pipeline for the Girona 500 underwater vehicle. The objective was to estimate the full AUV trajectory and reconstruct the scene observed by the downlooking camera during the sea-trial mission. We extended an existing non-incremental factor graph by integrating GTSAM-based structure and motion optimization with COLMAP-based feature matching, loop closure detection, sparse-model synchronization, and dense Poisson reconstruction.

The key contributions are: (1) formulation of the underwater SfM problem as a factor graph over 3,641 navigation poses with sparse visual factors on 279 COLMAP keyframes, (2) fusion of DVL/INS odometry with robust reprojection constraints (mean error reduced from 22.14 px to 0.47 px), and (3) round-trip export to COLMAP dense reconstruction yielding a mesh with 10.6×10^6 vertices. Hausdorff distance and F-score against a reference mesh remain future work when a ground-truth surface model is available.

Further improvements may include underwater image enhancement, stronger loop-closure validation, depth or bathymetric priors, and a fully automated ROSbag-to-mesh workflow on additional Girona 500 missions.

Acknowledgments

We thank our tutors Valerio Franchi and Nuno Gracias for their guidance and feedback. We also acknowledge the team involved in collecting and preparing the Girona 500 AUV sea-trial data used in this project.

References

[1] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2004.

[2] J. L. Schonberger and J.-M. Frahm, "Structure-from-motion revisited," *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4104–4113, 2016.

[3] J. L. Schonberger, E. Zheng, M. Pollefeys, and J.-M. Frahm, "Pixelwise view selection for unstructured multi-view stereo," *European Conference on Computer Vision (ECCV)*, pp. 501–518, 2016.

[4] F. Dellaert, "Factor graphs and GTSAM: A hands-on introduction," Georgia Institute of Technology, Technical Report, 2012.

[5] M. Kaess, H. Johannsson, R. Roberts, V. Ila, J. J. Leonard, and F. Dellaert, "iSAM2: Incremental smoothing and mapping using the Bayes tree," *International Journal of Robotics Research*, vol. 31, no. 2, pp. 216–235, 2012.

[6] N. Gracias and J. Santos-Victor, "Underwater video mosaics as visual navigation maps," *Computer Vision and Image Understanding*, vol. 79, no. 1, pp. 66–91, 2000.

[7] A. Mallios, P. Ridao, D. Ribas, F. Maurelli, and Y. Petillot, "EKF-SLAM for AUV navigation under probabilistic sonar scan-matching," *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4404–4411, 2010.

[8] D. Ribas, N. Palomeras, P. Ridao, M. Carreras, and A. Mallios, "Girona 500 AUV: From survey to intervention," *IEEE/ASME Transactions on Mechatronics*, vol. 17, no. 1, pp. 46–53, 2012.

[9] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.